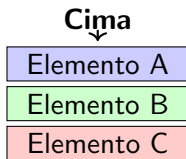
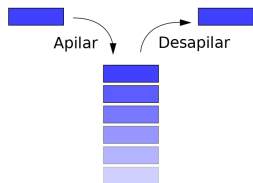


# Pilas y Colas (Implementación - Arrays y Listas Enlazadas)

Algoritmos y Estructuras de Datos

# Pilas (Tipo de Dato Abstracto - TAD)

- ▶ Una **pila** es una colección de elementos organizada linealmente con un extremo identificado como la **cima** o **tope**.
- ▶ Operaciones principales:
  - ▶ `push(data)`: Agrega un elemento en la cima.
  - ▶ `pop()`: Elimina y devuelve el elemento que se encuentra en la cima.
- ▶ **Política de acceso: LIFO** (*Last In, First Out*).



`push` → entra arriba

`pop` → sale el de arriba

# Operaciones de la Pila (TAD)

`make(tamaño)` Crea una pila vacía con tamaño máximo definido.

`push(data)` Inserta un elemento en la cima de la pila.

`pop()` Elimina el elemento en la cima y lo devuelve.

`top()` Devuelve el elemento en la cima sin eliminarlo.

`empty()` Devuelve `true` si está vacía, `false` en caso contrario.

# Pila - ¿Cómo podemos implementarla?

Analicemos un poco lo que conocemos

- ▶ ¿Se les ocurre como implementar?

# Pila - ¿Cómo podemos implementarla?

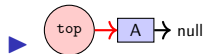
Analicemos un poco lo que conocemos

- ▶ ¿Se les ocurre como implementar?
- ▶ ¿Vamos pidiendo memoria y enlazando nodos?

# Pila - ¿Cómo podemos implementarla?

Analicemos un poco lo que conocemos

- ▶ ¿Se les ocurre como implementarla?
- ▶ ¿Vamos pidiendo memoria y enlazando nodos?



# Pila - ¿Cómo podemos implementarla?

Analicemos un poco lo que conocemos

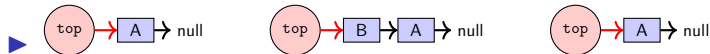
- ▶ ¿Se les ocurre como implementarla?
- ▶ ¿Vamos pidiendo memoria y enlazando nodos?



# Pila - ¿Cómo podemos implementarla?

Analicemos un poco lo que conocemos

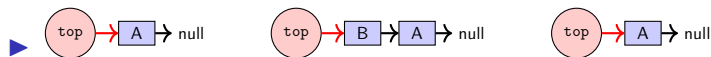
- ▶ ¿Se les ocurre como implementarla?
- ▶ ¿Vamos pidiendo memoria y enlazando nodos?



# Pila - ¿Cómo podemos implementarla?

Analicemos un poco lo que conocemos

- ▶ ¿Se les ocurre como implementarla?
- ▶ ¿Vamos pidiendo memoria y enlazando nodos?

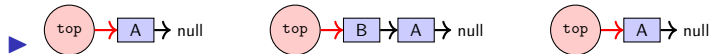


- ▶ ¿Tenemos alguna estructura de datos en la que nos podamos basar?

# Pila - ¿Cómo podemos implementarla?

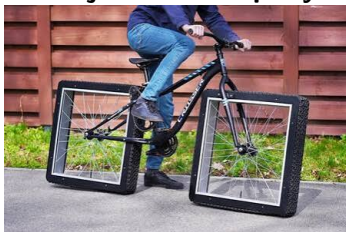
Analicemos un poco lo que conocemos

- ▶ ¿Se les ocurre como implementar?
- ▶ ¿Vamos pidiendo memoria y enlazando nodos?



- ▶ ¿Tenemos alguna estructura de datos en la que nos podamos basar?

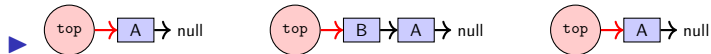
**¿No sería mejor usar las que ya existen?**



# Pila - ¿Cómo podemos implementarla?

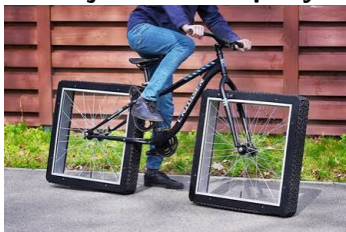
Analicemos un poco lo que conocemos

- ▶ ¿Se les ocurre como implementar?
- ▶ ¿Vamos pidiendo memoria y enlazando nodos?



- ▶ ¿Tenemos alguna estructura de datos en la que nos podamos basar?

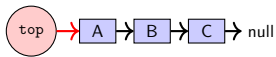
**¿No sería mejor usar las que ya existen?**



- ▶ ¿Qué ventaja nos daría esto?

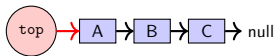
## Pila - Implementación con Lista Enlazada

- ▶ **Representación:** La cima de la pila es el **primer nodo** de la lista.
- ▶ **Nodo:** data y next.
- ▶ `push(data)`: Crea un nodo nuevo, lo enlaza al frente y actualiza la cabeza (`head`).
- ▶ `pop()`: Elimina y devuelve el primer nodo de la lista.
- ▶ **Eficiencia:** Solo se trabaja con el primer elemento, no se recorre la lista.



## Pila - Implementación con Lista Enlazada

- ▶ **Representación:** La cima de la pila es el **primer nodo** de la lista.
- ▶ **Nodo:** data y next.
- ▶ `push(data)`: Crea un nodo nuevo, lo enlaza al frente y actualiza la cabeza (head).
- ▶ `pop()`: Elimina y devuelve el primer nodo de la lista.
- ▶ **Eficiencia:** Solo se trabaja con el primer elemento, no se recorre la lista.



**Ventaja: No tiene límite de tamaño fijo (crece dinámicamente)**

# Implementación de Pilas sobre Arrays

Representación con índice 0

- ¿Cómo se podría implementar una pila sobre array?

# Implementación de Pilas sobre Arrays

Representación con índice 0

- ▶ ¿Cómo se podría implementar una pila sobre array?
- ▶ **Representación:** A (array), top (int).

# Implementación de Pilas sobre Arrays

Representación con índice 0

- ▶ ¿Cómo se podría implementar una pila sobre array?
- ▶ **Representación:** A (array), top (int).
- ▶ **Y sabemos que deberá cumplir:**

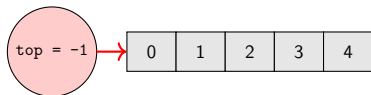
$$-1 \leq R.top < |R.A|$$

# Implementación de Pilas sobre Arrays

Representación con índice 0

- ▶ ¿Cómo se podría implementar una pila sobre array?
- ▶ **Representación:** A (array), top (int).
- ▶ **Y sabemos que deberá cumplir:**

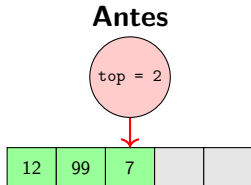
$$-1 \leq R.top < |R.A|$$



Índices:

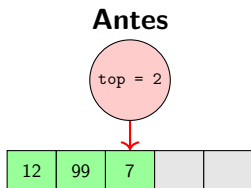
# Operaciones sobre Arrays (índice 0)

Ejemplo de push



# Operaciones sobre Arrays (índice 0)

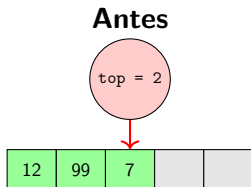
Ejemplo de push



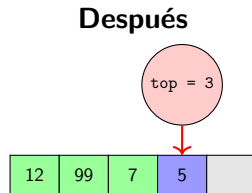
→  
push(5)

# Operaciones sobre Arrays (índice 0)

Ejemplo de push

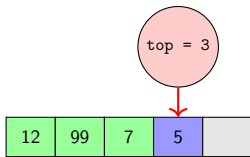


→  
push(5)



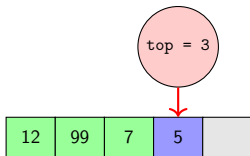
# Operaciones sobre Arrays (índice 0)

Ejemplo de pop

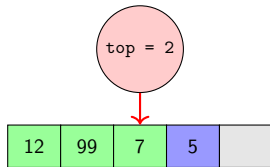


# Operaciones sobre Arrays (índice 0)

Ejemplo de pop

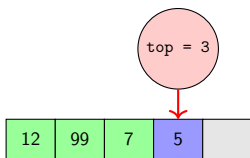


**Ejemplo:** `pop()` → devuelve 5 y top baja a 2

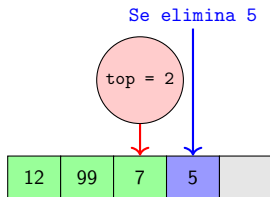


# Operaciones sobre Arrays (índice 0)

Ejemplo de pop

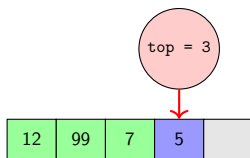


**Ejemplo:** `pop()` → devuelve 5 y top baja a 2

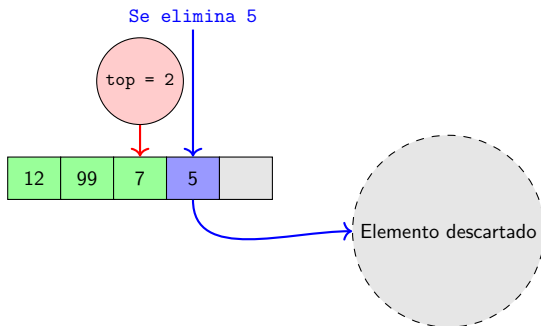


# Operaciones sobre Arrays (índice 0)

Ejemplo de pop



**Ejemplo:** `pop()` → devuelve 5 y top baja a 2



## Algunas características de Pilas con Arrays

- ▶ **Todas las operaciones son “rápidas”:** Nunca se recorre el array completo (make, push, pop, top, etc)

## Algunas características de Pilas con Arrays

- ▶ **Todas las operaciones son “rápidas”:** Nunca se recorre el array completo (make, push, pop, top, etc)
- ▶ **Limitación:** Tamaño limitado al array. Si se llena, se produce overflow.

## Colas (Tipo de Dato Abstracto - TAD)

- ▶ Una **cola** es una colección lineal con dos extremos:
  - ▶ **Frente (Inicio)**: Donde se remueven los elementos.
  - ▶ **Final (Fin)**: Donde se insertan nuevos elementos.
- ▶ Operaciones principales:
  - ▶ `enqueue(data)`: Agrega un elemento al final de la cola.
  - ▶ `dequeue()`: Remueve y devuelve el elemento al frente.
- ▶ **Política de acceso: FIFO** (*First In, First Out*).



# Implementación de Cola con Lista Enlazada

Dos punteros: `head` y `tail`

Implementemos con lista enlazada

# Implementación de Cola con Lista Enlazada

Dos punteros: `head` y `tail`

Implementemos con lista enlazada

- **Representación:** Dos punteros:

# Implementación de Cola con Lista Enlazada

Dos punteros: `head` y `tail`

Implementemos con lista enlazada

- ▶ **Representación:** Dos punteros:
  - ▶ `head`: primer nodo (frente).

# Implementación de Cola con Lista Enlazada

Dos punteros: `head` y `tail`

Implementemos con lista enlazada

- ▶ **Representación:** Dos punteros:
  - ▶ `head`: primer nodo (frente).
  - ▶ `tail`: último nodo (final).

# Implementación de Cola con Lista Enlazada

Dos punteros: `head` y `tail`

Implementemos con lista enlazada

- ▶ **Representación:** Dos punteros:
  - ▶ `head`: primer nodo (frente).
  - ▶ `tail`: último nodo (final).
- ▶ `enqueue(data)`: Se crea un nodo, se enlaza al final y se actualiza `tail`

# Implementación de Cola con Lista Enlazada

Dos punteros: `head` y `tail`

Implementemos con lista enlazada

- ▶ **Representación:** Dos punteros:
  - ▶ `head`: primer nodo (frente).
  - ▶ `tail`: último nodo (final).
- ▶ `enqueue(data)`: Se crea un nodo, se enlaza al final y se actualiza `tail`

# Implementación de Cola con Lista Enlazada

Dos punteros: `head` y `tail`

Implementemos con lista enlazada

- ▶ **Representación:** Dos punteros:
  - ▶ `head`: primer nodo (frente).
  - ▶ `tail`: último nodo (final).
- ▶ `enqueue(data)`: Se crea un nodo, se enlaza al final y se actualiza `tail` ¿Cuánto recorre?.
- ▶ `dequeue()`: Se elimina el nodo apuntado por `head` y se actualiza la cabeza

# Implementación de Cola con Lista Enlazada

Dos punteros: `head` y `tail`

Implementemos con lista enlazada

- ▶ **Representación:** Dos punteros:
  - ▶ `head`: primer nodo (frente).
  - ▶ `tail`: último nodo (final).
- ▶ `enqueue(data)`: Se crea un nodo, se enlaza al final y se actualiza `tail` ¿Cuánto recorre?.
- ▶ `dequeue()`: Se elimina el nodo apuntado por `head` y se actualiza la cabeza

# Implementación de Cola con Lista Enlazada

Dos punteros: `head` y `tail`

Implementemos con lista enlazada

- ▶ **Representación:** Dos punteros:
  - ▶ `head`: primer nodo (frente).
  - ▶ `tail`: último nodo (final).
- ▶ `enqueue(data)`: Se crea un nodo, se enlaza al final y se actualiza `tail` ¿Cuánto recorre?.
- ▶ `dequeue()`: Se elimina el nodo apuntado por `head` y se actualiza la cabeza ¿Cuánto recorre?.
- ▶ Dinámica y sin límite de tamaño.

# Implementación de Cola con Lista Enlazada

Dos punteros: `head` y `tail`

Implementemos con lista enlazada

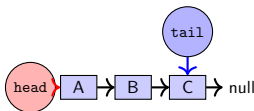
- ▶ **Representación:** Dos punteros:
  - ▶ `head`: primer nodo (frente).
  - ▶ `tail`: último nodo (final).
- ▶ `enqueue(data)`: Se crea un nodo, se enlaza al final y se actualiza `tail` ¿Cuánto recorre?.
- ▶ `dequeue()`: Se elimina el nodo apuntado por `head` y se actualiza la cabeza ¿Cuánto recorre?.
- ▶ Dinámica y sin límite de tamaño.

# Implementación de Cola con Lista Enlazada

Dos punteros: `head` y `tail`

Implementemos con lista enlazada

- ▶ **Representación:** Dos punteros:
  - ▶ `head`: primer nodo (frente).
  - ▶ `tail`: último nodo (final).
- ▶ `enqueue(data)`: Se crea un nodo, se enlaza al final y se actualiza `tail` ¿Cuánto recorre?.
- ▶ `dequeue()`: Se elimina el nodo apuntado por `head` y se actualiza la cabeza ¿Cuánto recorre?.
- ▶ Dinámica y sin límite de tamaño.



## Operaciones de la Cola (TAD)

`make(tamaño)` Crea una cola vacía con un array de tamaño fijo.

`enqueue(dato)` Inserta un elemento al final.

`dequeue()` Elimina y devuelve el elemento al frente.

`head()` Devuelve el elemento al frente sin eliminarlo.

`empty()` Devuelve `true` si está vacía, `false` en caso contrario.

## Operaciones de la Cola (TAD)

`make(tamaño)` Crea una cola vacía con un array de tamaño fijo.

`enqueue(dato)` Inserta un elemento al final.

`dequeue()` Elimina y devuelve el elemento al frente.

`head()` Devuelve el elemento al frente sin eliminarlo.

`empty()` Devuelve true si está vacía, false en caso contrario.

**¿Podríamos implementarla sobre un array?**

# Operaciones de la Cola (TAD)

`make(tamaño)` Crea una cola vacía con un array de tamaño fijo.

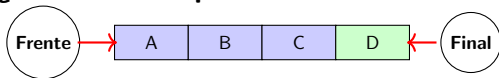
`enqueue(dato)` Inserta un elemento al final.

`dequeue()` Elimina y devuelve el elemento al frente.

`head()` Devuelve el elemento al frente sin eliminarlo.

`empty()` Devuelve true si está vacía, false en caso contrario.

**¿Podríamos implementarla sobre un array?**



`dequeue()` → sale A — `enqueue(D)` → entra por Final

# Operaciones de la Cola (TAD)

`make(tamaño)` Crea una cola vacía con un array de tamaño fijo.

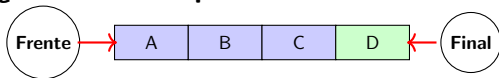
`enqueue(dato)` Inserta un elemento al final.

`dequeue()` Elimina y devuelve el elemento al frente.

`head()` Devuelve el elemento al frente sin eliminarlo.

`empty()` Devuelve true si está vacía, false en caso contrario.

**¿Podríamos implementarla sobre un array?**



`dequeue()` → sale A — `enqueue(D)` → entra por Final

¿Cómo hacemos para avanzar y retroceder? ¿Qué pasa si se llena?

# Operaciones de la Cola (TAD)

`make(tamaño)` Crea una cola vacía con un array de tamaño fijo.

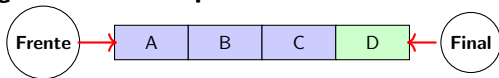
`enqueue(dato)` Inserta un elemento al final.

`dequeue()` Elimina y devuelve el elemento al frente.

`head()` Devuelve el elemento al frente sin eliminarlo.

`empty()` Devuelve true si está vacía, false en caso contrario.

**¿Podríamos implementarla sobre un array?**



`dequeue()` → sale A — `enqueue(D)` → entra por Final

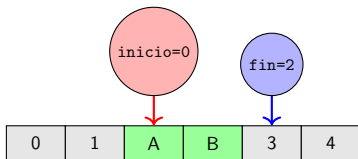
¿Cómo hacemos para avanzar y retroceder? ¿Qué pasa si se llena?

Podríamos usar un buffer circular

# Implementación de Colas con Arrays

Búfer Circular (índice 0)

- ▶ **Representación:**  $Q$  (array), inicio, fin (int).
- ▶ **Invariante práctico:**
  - ▶ **Vacía:**  $\text{inicio} == \text{fin}$
  - ▶ **Llena:**  $(\text{fin} + 1) \% |Q| == \text{inicio}$



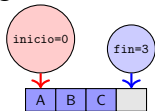
**Nota: Espacio de seguridad**

# Ambigüedad al llenar la cola circular

¿Cómo distinguimos entre “lleno” y “vacío” si no dejamos espacio?

## Antes de enqueue(D)

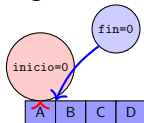
(Sin ambigüedad:  $\text{inicio} \neq \text{fin}$ )



Estado válido:  $\text{inicio} \neq \text{fin}$

## Después de enqueue(D)

(¡Se genera ambigüedad!:  $\text{inicio} = \text{fin}$ )



¡Ambiguo!  
 $\text{inicio} == \text{fin}$

**Solución:** Dejar un espacio vacío. La condición de lleno será  $(\text{fin} + 1) \% |Q| == \text{inicio}$ . Así nunca se igualan.

# Operaciones sobre Arrays (índice 0)

enqueue y dequeue

## **Encolar** (enqueue(Q, x)):

- ▶ SI isFull(Q): ERROR  
  ‘‘overflow’’
- ▶ SINO:
  - ▶  $Q[Q.fin] = x$
  - ▶  $Q.fin = (Q.fin + 1) \% |Q|$

## **Desencolar** (dequeue(Q)):

- ▶ SI empty(Q): ERROR  
  ‘‘underflow’’
- ▶ SINO:
  - ▶  $x = Q[Q.inicio]$
  - ▶  $Q.inicio = (Q.inicio + 1) \% |Q|$
  - ▶ RETURN x

# Operaciones sobre Arrays (índice 0)

enqueue y dequeue

## Encolar (enqueue(Q, x)):

- ▶ SI isFull(Q): ERROR  
  ‘‘overflow’’
- ▶ SINO:
  - ▶  $Q[Q.fin] = x$
  - ▶  $Q.fin = (Q.fin + 1) \% |Q|$

## Desencolar (dequeue(Q)):

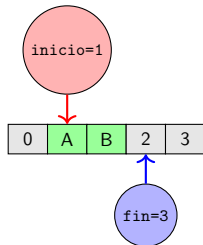
- ▶ SI empty(Q): ERROR  
  ‘‘underflow’’
- ▶ SINO:
  - ▶  $x = Q[Q.inicio]$
  - ▶  $Q.inicio = (Q.inicio + 1) \% |Q|$
  - ▶ RETURN x

Notar que no hay ciclos, es decir no hace falta recorrer la estructura.  
Es **eficiente**.

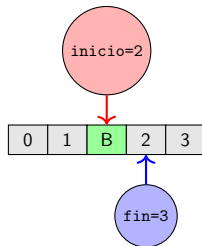
# Ejemplo de Búfer Circular (Índice 0)

Movimiento de inicio y fin

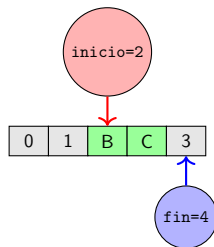
**Estado Inicial**



**Después de dequeue**



**Después de enqueue(C)**



¡A codear!

